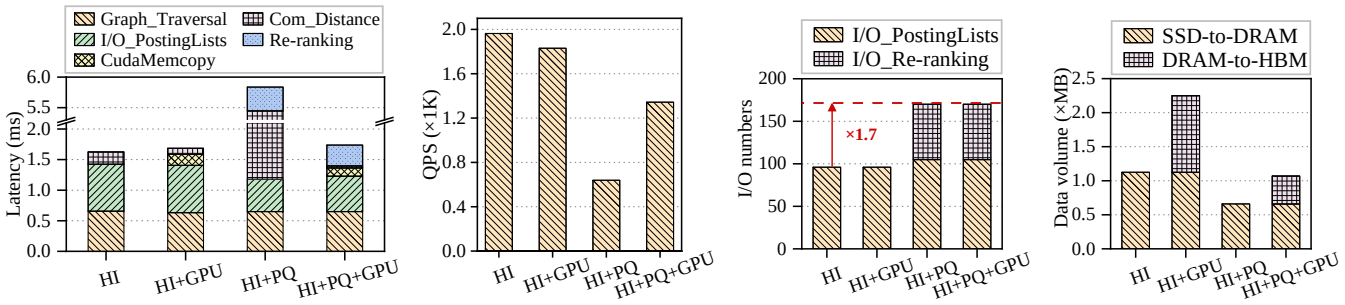
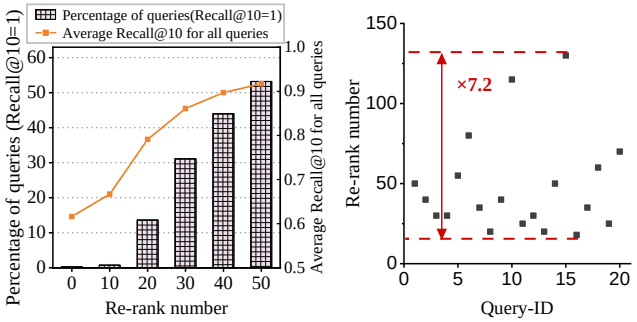


(a) Latency per query (b) Throughput (c) I/O numbers per query (d) Data transmission per query

Figure 4: Three combinations of *hierarchical indexing* (HI), *product quantization* (PQ), and GPU acceleration



(a) Accuracy vs. re-ranking numbers (b) Re-ranking number per query

Figure 5: Differential characterization between queries

Challenge 1: Overall, the combinations of HI, PQ, and GPU acceleration techniques cause even higher latency and lower throughput than SPANN that adopts HI solely. The root cause is that the GPU's HBM still cannot accommodate all posting lists compressed by PQ, allowing extensive data transmission between GPU and CPUs to become a new performance bottleneck. **Without a sophisticated design of the data layout across different devices and a careful collaboration among these three techniques, it is impossible to fully realize the GPU's potential for ANNS acceleration**.

Challenge 2: To achieve the same level of query accuracy, a re-ranking process is usually required to refine intermediate results generated by the GPU. The number of the top-n vectors that should be re-ranked (i.e., the re-ranking number) is usually several times larger than the final top-k nearest neighbors. To evaluate the impact of the re-ranking number on the query accuracy, we execute 10,000 queries by linearly increasing the re-ranking number. As shown in Figure 5a, when the re-ranking number is set to 40, about 42% of queries get the accurate top-10 nearest neighbors under $Recall@10 = 1.0$, while all queries achieve an accuracy level[1] of $Recall@10 = 0.9$ on average. In this case, it is only beneficial to increase the re-ranking number for straggler queries. Moreover, we find that the minimum re-ranking numbers are very distinct for different ANNS queries, as shown in Figure 5b. This significant variance usually causes unnecessary I/O operations and distance calculations if the number of re-ranked vectors is fixed for all queries. However, **it is challenging to determine the minimum re-ranking number for each query under a given accuracy constraint**.

Challenge 3: The re-ranking process introduces a number of I/O requests to raw vectors on SSDs. The size of a raw vector generally ranges from 128 to 384 bytes, while the smallest operating unit of modern NVMe SSDs is typically a page (4 KB). **This mismatch in granularity causes significant read amplification, resulting in extremely low I/O efficiency during re-ranking.** Fortunately, we find these vectors requiring re-ranking usually are highly similar to each other. This similarity offers an opportunity to mitigate the read amplification by reorganizing the data layout on SSDs.

3 FusionANNS Design

In this section, we present key designs of FusionANNS, i.e., multi-tiered indexing, CPU/GPU collaborative filtering, heuristic re-ranking, and redundant-aware I/O deduplication.

3.1 Multi-tiered Indexing

Figure 6 illustrates the structure of multi-tiered indices residing in host main memory, GPU's HBM, and SSDs. We construct multi-tiered indices in an offline manner. At first, we adopt the hierarchical balanced clustering algorithm [34] to iteratively partition the dataset into several posting lists. Each posting list contains multiple vector-IDs and the corresponding vector content. We follow SPANN to configure the number of posting lists (about 10% of the total number of vectors in a datasets) and use the same replication mechanism to address the boundary concern [15]. Specifically, when a vector lies on the boundary of multiple clusters, we assign this boundary vector to a cluster according to Equation 2.

$$v \in C_i \Leftrightarrow \text{Dist}(v, C_i) \leq (1+\varepsilon) \times \text{Dist}(v, C_1) \qquad (2)$$

where v represents the vector to be assigned, C_i denotes the i-th clusters. Particularly, C_1 represents the cluster that is clos-

[1] $Recall@k$ refers to the proportion of relevant vectors that are retrieved within the ground-truth top k nearest neighbors.

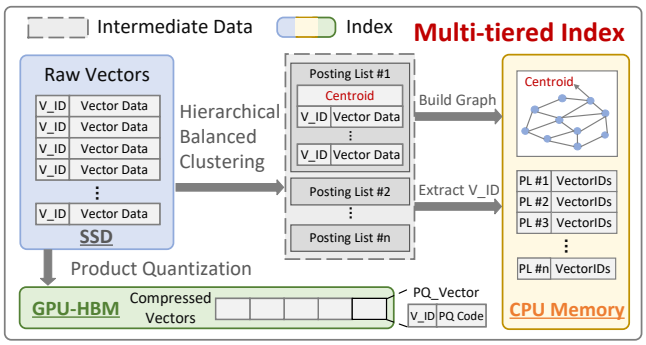


Figure 6: Mutil-tiered indices in FusionANNS

est to the vector v. The parameter ε determines the maximum distance in which a vector should be assigned simultaneously to multiple clusters. To balance the query accuracy and efficiency, each vector is assigned to eight clusters at most [15].

In-memory Indices. After the dataset is clustered, we build a graph index based on SPTAG [36] using the centroids of all posting lists and store it in main memory. With this navigation graph, FusionANNS can efficiently identify the top-m nearest posting lists for a query vector. This graph is constructed by continuously adding new vectors to an empty graph. When a vector is added as a new vertex, new edges are created to connect this newly-added vertex with its top-k (typically 64) nearest neighbors. Then, its neighboring vertices should update their nearest neighbors to limit the maximum number of edges. Unlike SPANN that stores all posting lists on SSDs, we extract only vector-IDs of each posting list as metadata (excluding vector content) and store it in memory, as shown in Figure 6. When the graph index and metadata are generated, the intermediate posting lists can be discarded. Since the memory footprint of the graph and metadata is relatively small, FusionANNS can support billion-scale ANNS in a memory cost-efficient way using general-purpose servers.

PQ-based Vectors in GPU's HBM. Since PQ can significantly reduce the memory footprint of high-dimensional vectors via lossy-compression, even an entry-level GPU such as NVIDIA V100 with 32 GB HBM can accommodate all compressed vectors in its HBM for billion-scale datasets. In FusionANNS, we pin all compressed vectors in the HBM, avoiding extensive data swapping between GPUs and CPUs that is commonly experienced in previous GPU-accelerated ANNS systems [9, 20, 24]. Since in-memory indices still remain the benefit of the replication mechanism for boundary vectors, FusionANNS can efficiently obtain all IDs of candidate vectors, and then sends these vector-IDs (excluding vectors' content) to the GPU for distance calculations. In this way, FusionANNS also eliminates the performance bottleneck caused by the limited PCIe bandwidth between CPUs and GPUs.

Raw Vectors on SSDs. Unlike IVF-based SPANN that stores all posting lists on SSDs, FusionANNS only needs to store raw vectors on SSDs for re-ranking. Since the volume of raw vectors is almost 8 times smaller than that of posting lists, FusionANNS can significantly reduce the storage consumption. For each query, since only the re-ranking process lead to a few I/O requests, FusionANNS can also alleviate the I/O bottleneck of SSDs for concurrent queries.

Remarks. Previous SSD-based ANNS systems such as SPANN [15] and DiskANN [22] cannot utilize GPU to accelerate ANNS since GPU cannot reduce I/O requests for each ANNS query but introduces significant performance overhead of data movement. In contrast, our multi-tiered indexing approach can significantly reduce the storage footprints on HBM, main memory, and SSDs. It also significantly reduces the amount of data transferred across SSDs, CPUs, and GPUs.

3.2 CPU/GPU Collaborative Filtering

The multi-tiered indices enable CPU/GPU collaborative filtering for ANNS queries. Figure 7 illustrates the system architecture of FusionANNS and the workflow of an ANNS query. Upon a vector query, FusionANNS first utilizes the GPU to generate the query vector's distance table for subsequent PQ distance calculations (❶). Meanwhile, the CPU traverses the in-memory navigation graph to identify the top-m posting lists nearest to the query vector (❷). Then, the CPU consults the metadata to collect vector-IDs within these candidate posting lists (❸). After that, the CPU transfers these vector-IDs to the GPU and invokes GPU kernels (❹) for further processing.

When the GPU receives vector-IDs, it first deduplicates them using a parallel hash module (❺). For each vector-ID, the GPU reads the corresponding PQ vector from HBM and computes the PQ distance between it and the query vector. In this step, the GPU allocates a thread for each dimension of the PQ vector to access the corresponding precomputed value in the distance table. Then, a coordinator thread accumulates these values to count the PQ distance (❻) for each candidate vector. Subsequently, the GPU sorts all distances in ascending order and returns the top-n vectors' ID to the CPU (❼). Since the intermediate results obtained using PQ vectors are not precise, the CPU reads the raw vectors according to these vector-IDs from SSDs for further re-ranking (❽). Finally, the CPU returns the final top-k nearest neighbors.

Remarks. This CPU/GPU collaborative filtering mechanism can filter out most irrelevant vectors throughput two rounds of retrieving, and thus can significantly reduce the number of I/O requests to SSDs during the re-ranking stage.

3.3 Heuristic Re-ranking

Since PQ causes an accuracy loss during distance calculations, a re-ranking process is usually required to refine the intermediate results reported by the GPU. As mentioned in Section 2.3, to achieve the same level of query accuracy, the minimum re-ranking numbers for different ANNS queries usually vary significantly. Thus, a static configuration of the

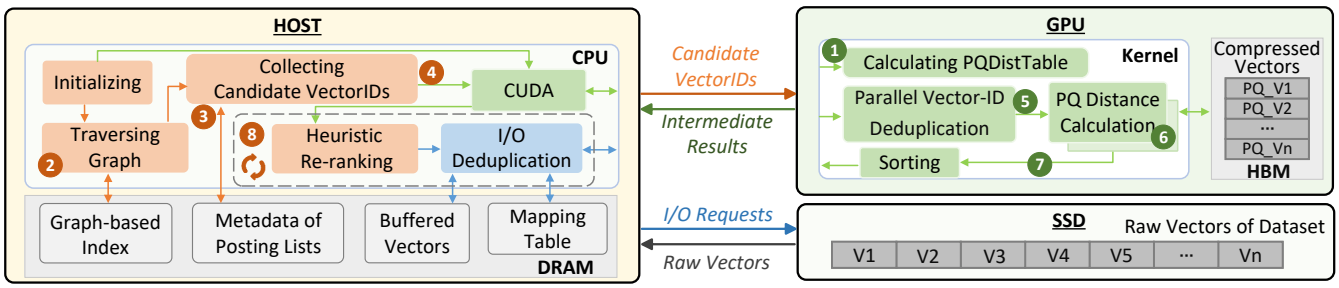


Figure 7: The FusionANNS architecture and the workflow of an ANNS query

Algorithm 1: Heuristic Re-ranking

Input: $Tasks, BatchSize, k, \varepsilon, \beta$
Output: Q

1 *Initialize* $Q \leftarrow NULL$
2 *Initialize* $StabilityCounter \leftarrow 0$
3 **for** $i = 0; i < Tasks.size(); i += BatchSize$ **do**
4 $S_{n-1} \leftarrow Q.GetVectorIDs()$
5 **for** $j = i; j < i + BatchSize; j++$ **do**
/* Tasks[j] involves I/Os and computations */
6 $Candidate\_Vector \leftarrow GetDistance(Tasks[j])$
7 $Q.insert(Candidate\_Vector)$
8 **end**
9 $S_n \leftarrow Q.GetVectorIDs()$
10 $\Delta \leftarrow \frac{|S_n - S_n \cap S_{n-1}|}{k}$ /* Calculating change rate Δ */
11 **if** $\Delta \leq \varepsilon$ **then**
12 $StabilityCounter \leftarrow StabilityCounter + 1$
13 **if** $StabilityCounter \geq \beta$ **then**
14 **return** Q /* Terminate re-ranking */
15 **end**
16 **else**
17 $StabilityCounter \leftarrow 0$
18 **end**
19 **end**
20 **return** Q

re-ranking number may cause unnecessary I/O operations and distance calculations, or results in an accuracy loss. Previous in-memory ANNS systems such as LanceDB [37] re-rank the same number of candidate vectors for simplicity because the fast memory can tolerate some unnecessary data accesses. However, this simple approach is inefficient for SSD-based ANNS systems because massive-yet-small I/O requests initiated by concurrent ANNS queries can cause significant performance degradation due to the limited IOPS of SSDs.

To circumvent this problem, we propose a *heuristic re-ranking* mechanism to minimize I/O operations and distance calculations. The key idea is to set a relative large re-ranking number conservatively for high accuracy, and to terminate the re-ranking process immediately once the subsequent search is no longer beneficial for improving the query accuracy. To achieve this goal, we divide the re-ranking process into multiple mini-batches and execute them sequentially. Each mini-batch contains the same number of candidate vectors. Since all candidate vectors are sorted with their distances in ascending order, the mini-batch executed earlier usually has a higher possibility to identify more vectors that belong to the final top-k nearest neighbours. Once a mini-batch is finished, we exploit a lightweight feedback control model to check whether subsequent mini-batches are beneficial for improving the query accuracy.

To simplify the problem, we use a priority queue Q (i.e., a max-heap) to maintain the current top-k nearest neighbours. Initially, the max-heap is empty. For each mini-batch, we calculate the distances between the query vector and vectors within this mini-batch, and insert the vector whose distance is less than the current maximum distance in the max-heap. When a mini-batch is finished, we calculate the change rate of the max-heap according to Equation 3:

$$\Delta = \frac{|S_n - S_n \cap S_{n-1}|}{k} \tag{3}$$

where S_n and S_{n-1} represent the sets of vectors' IDs in the max-heap when the mini-batch n and the mini-batch n-1 are just completed, respectively. k represents the number of vectors maintained in the max-heap, i.e., the number of the final nearest neighbors. We terminate the re-ranking process if the change rate of the max-heap for successive mini-batches is smaller than a given threshold ε for β times continuously.

Algorithm 1 presents the pseudo-code for the heuristic re-ranking. We first initialize the max-heap Q as NULL, and use a *StabilityCounter* to record the times that the change rate remains lower than ε continuously. The size of *Tasks* denotes the total number of vectors should be re-ranked in-batch. The parameter *BatchSize* denotes the number of candidate vectors in a mini-batch. For each mini-batch, we retrieve the top-k vectors' IDs from Q before processing tasks in this mini-batch (line 4). Then, we sequentially perform each task including reading the raw vector from SSDs, calculating its distance to the query vector, and inserting this vector into Q if its distance is less than the maximum value in the max-heap. When a mini-batch is finished, we collect the IDs of updated top-k vectors from Q, and calculate the change rate Δ of Q between

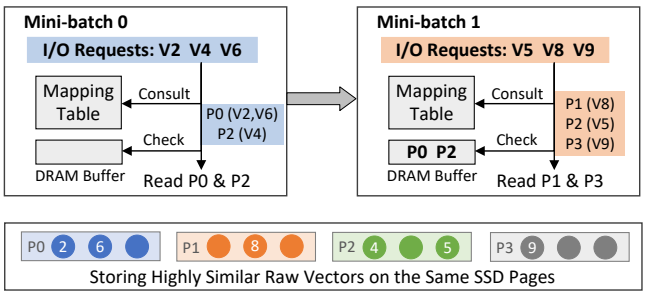


Figure 8: Optimized data layout and I/O deduplication

these two successive mini-batches (line 9-10). If the change rate is lower than the given threshold ε, we increase the *StabilityCounter* by one. Once the *StabilityCounter* becomes larger than the given threshold β, the re-ranking process is terminated. In contrast, if the change rate exceeds the threshold ε, we reset the *StabilityCounter* and continue the following mini-batches. At last, we return the final top-k vectors in Q. To achieve the optimal performance, we set ε, β, and *BatchSize* as 0.1, 1, and k, respectively according to our experimental studies.

Remarks. Our heuristic re-ranking algorithm can minimize I/O requests to SSDs and CPU resource consumption for distance calculation while guaranteeing high accuracy, and eventually reduces the latency of re-ranking.

3.4 Redundant-aware I/O Deduplication

FusionANNS uses raw vectors on SSDs to re-rank the intermediate results returned by the GPU. A straightforward approach is to store all raw vectors sequentially on SSD pages. However, since a raw vector (128∼384 bytes) is quite smaller than the page granularity (4KB), individual requests to these raw vectors often result in significant read amplification. Moreover, since the re-ranking process introduces a lot of random and small I/O operations, the I/O latency has a crucial impact on the end-to-end query latency. Like most SSD-based ANNS systems [15], we adopt Direct I/O [38, 39] to fully exploit the low-latency property of modern NVMe SSDs. To further improve I/O efficiency, we first optimize the data layout to improve the spatial locality on SSDs. Then, we exploit *redundancy-aware I/O deduplication* to mitigate the effect of read amplification.

Optimized Storage Layout. Although the vectors requiring re-ranking are obtained by PQ distances, they are highly similar to the query vector, allowing them usually spatially close to each other. This similarity offers an opportunity to mitigate the read amplification by carefully organizing the data layout on SSDs. Specifically, when the in-memory indices are created offline, for each centroid in the navigation graph, we use a bucket to store a number of raw vectors that are closest to the centroid. We note that there are not duplicate vectors among buckets. For each bucket, if it does not align with SSD pages, we combine buckets based on the size of unaligned portions using a max-min algorithm [40] to minimize the free space on a SSD page. Finally, we group all buckets as a single file and store it on SSDs, and use a table in memory to maintain the mappings between vectors and SSD pages.

Intra- and Inter- Mini-batch I/O Deduplication. Empowered by the optimized data layout, we design two I/O deduplication mechanisms, including merging I/Os mapped to the same SSD page within a mini-batch, and exploiting the DRAM buffer to eliminate redundant I/Os in subsequent mini-batches. Here, we use a simple example to describe these mechanisms, as shown in Figure 8. Assume that there are two mini-batches in the re-ranking process, where the tasks of *mini-batch 0* is to re-rank vectors: *V2*, *V4*, and *V6*. The tasks of *mini-batch 1* is to re-rank vectors:*V5*, *V8*, and *V9*. When *mini-batch 0* is executed, it first consults the mapping table to obtain the SSD page-IDs corresponding to the requested vectors. Since both *V2* and *V6* are stored in the same SSD page *P0*, we can merge these two I/O requests and only read one SSD page to get *V2* and *V6*. Since *P0* and *P2* do not exist in the DRAM buffer, we directly read them to the DRAM buffer via two I/O requests. In *mini-batch 1*, although *V5*, *V8*, and *V9* are stored in different SSD pages, the DRAM buffer already contains *P2* which includes *V5*. Therefore, *mini-batch 1* only needs to read *P1* and *P3* via two I/O requests.

Remarks. We optimize the data layout on SSDs to enable intra- and inter-mini-batch I/O deduplication mechanisms, which eventually mitigate the effect of read amplification and improve I/O efficiency.

4 Implementation

We implement the system prototype of FusionANNS using 22K lines of codes in C++ and CUDA. FusionANNS can be widely deployed in general-purpose servers equipped with an entry-level GPU. Like most ANNS systems [14, 15, 22, 41], we use each CPU thread to handle an individual query.

Contention-free GPU Memory Management. FusionANNS implements a GPU memory manager specifically for concurrent ANNS queries. During system initialization, we first load compressed vectors into GPU's HBM, and use the remaining space as a memory pool. Then, we divide the memory pool into several independent blocks, each of which is assigned to a single query as working memory. Once a query is finished, its block can be assigned to other pending queries. This approach can avoid frequent memory allocations and lock contention between queries, improving the system performance.

Efficient GPU Kernels. For each vector, we allocate multiple GPU threads according to their dimensions to calculate distances in parallel. Moreover, we design a kernel to support parallel deduplication of vector-IDs using a hash algorithm. For a list of candidate Vector-IDs, we allocate a GPU thread for each vector-ID and use a spinlock to ensure that only one thread can access and update a hash table entry at a time.